
Project Title and Student Details



Project Overview

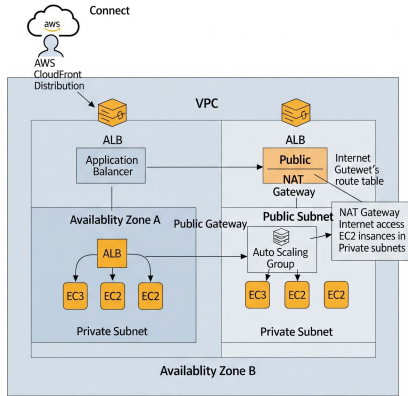
Scalable AWS Cloud Architecture implemented with Terraform for infrastructure as code, focusing on high availability and low latency.



Student & Course Information

Student: Vishrut Pandey, Course: Cloud Programming (DLBSEPCP01_E), University: IU International University of Applied Sciences, Date: 24th June 2024.

Final Architecture Diagram



Comprehensive AWS Architecture

Component Interaction Overview

This diagram illustrates the complete AWS environment, showing segmented VPC with public and private subnets, NAT gateways, security groups, EC2 Auto Scaling group, and an Application Load Balancer directing traffic.

Traffic flows from CloudFront CDN to the Application Load Balancer, which routes requests to EC2 instances in private subnets; Terraform automates deployment ensuring consistency and scalability.

Technology Stack Overview



AWS Cloud Services

AWS provides core infrastructure components including VPC, EC2, Auto Scaling, Load Balancer, and CloudFront CDN for building a resilient and scalable web application.



Terraform Infrastructure as Code

Implemented infrastructure provisioning and management using Terraform to enable repeatable, consistent deployments and versioned infrastructure definitions.



GitHub Source Control

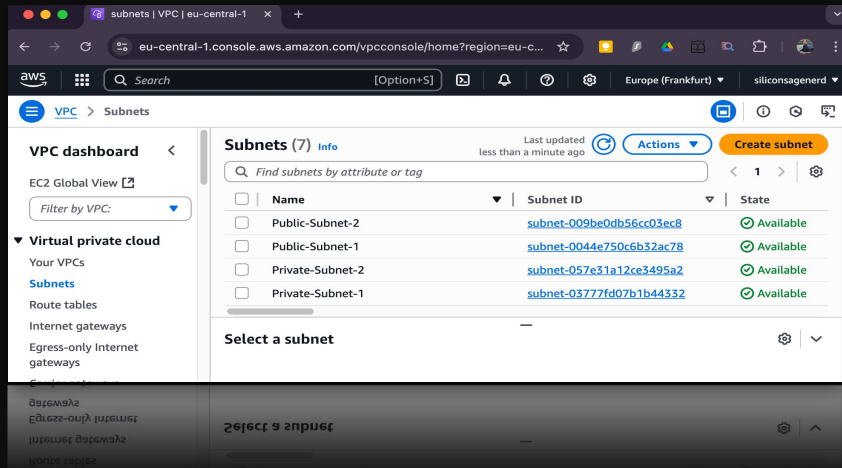
GitHub manages source code and Terraform configurations with version control, facilitating collaboration and tracking changes to the infrastructure codebase over time.

VPC and Subnet Setup with Terraform

```
network.tf
2 # Defines the main Virtual Private Cloud
3 resource "aws_vpc" "main" {
4   cidr_block = var.vpc_cidr
5   tags = {
6     Name = "Main-VPC"
7   }
8 }
9
10 # Creates the public subnets across different Availability Zones
11 resource "aws_subnet" "public" {
12   count           = length(var.public_subnet_cidrs)
13   vpc_id         = aws_vpc.main.id
14   cidr_block     = var.public_subnet_cidrs[count.index]
15   availability_zone = tolist(data.aws_availability_zones.available.names)[count.index]
16   map_public_ip_on_launch = true # Important for public subnets
17
18   tags = {
19     Name = "Public-Subnet-${count.index + 1}"
20   }
21 }
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Building the Foundation: Secure Networking

- The Virtual Private Cloud (VPC) was configured to provide an isolated network environment, using a CIDR block of 10.0.0.0/16 to allow ample IP address space for scaling.
- Public and private subnets were created across multiple Availability Zones to ensure high availability and fault tolerance. Public subnets host resources accessible from the internet, while private subnets contain internal resources.
- Terraform code declares the VPC resource and subnet blocks with explicit CIDR ranges, enabling repeatable and version-controlled infrastructure deployment.
- AWS Console screenshots verify the successful creation of the VPC and subnets, illustrating their correct CIDR ranges and Availability Zone placement.



Routing and Security Configuration

The screenshot displays the AWS Management Console interface for Security Groups. The top navigation bar shows the AWS logo, a search bar, and the region 'Europe (Frankfurt)'. The main content area is titled 'Security Groups (1/4)' and includes a search bar and buttons for 'Actions', 'Export security groups to CSV', and 'Create security group'. A table lists four security groups:

Name	Security group ID	Security group name	VPC ID	Description	Owner	Inbound rules count
EC2-Security-Group	sg-08e9a227d58baee36	ec2-security-group	vpc-0ea4f01e9728a05a	Allow HTTP from the ALB	248547463652	1 Permission entry
-	sg-05288c20c4c05999	default	vpc-0ea4f01e9728a05a	default VPC security group	248547463652	1 Permission entry
-	sg-9d58a6cf12e607f50	default	vpc-00a8202aa79a350ac7	default VPC security group	248547463652	1 Permission entry
ALB-Security-Group	sg-076ea483a29f68f5aa	alb-security-group	vpc-0ea4f01e9728a05a	Allow HTTP and HTTPS inbound traffic ...	248547463652	2 Permission entries

Below the table, the 'Inbound rules' tab is selected for the 'sg-08e9a227d58baee36 - ec2-security-group'. It shows a single inbound rule:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sg-0fb1dc37141b3d879	-	HTTP	TCP	80	sg-076ea483a29f68f5aa	HTTP from ALB

Key Points

- **Internet Gateway (IGW):** Enables communication between the public subnets in our VPC and the internet, allowing users to reach our Application Load Balancer.
- **NAT Gateway:** This is placed in a public subnet and provides a secure, one-way path for instances in the private subnets to download software and updates from the internet.
- **Security Groups:** These act as virtual firewalls controlling traffic to our resources. The configuration ensures that only the Application Load Balancer can send web traffic to our EC2 instances, creating a secure backend.

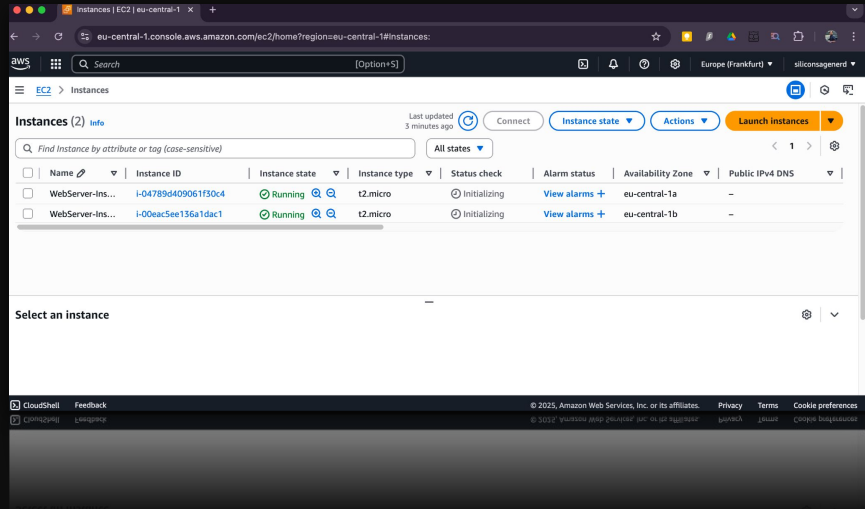
Compute Layer: EC2 and Auto Scaling

```
compute.tf
# Auto Scaling Group
resource "aws_autoscaling_group" "web_asg" {
  name                = "web-server-asg"
  desired_capacity     = 2
  max_size            = 4
  min_size            = 2
  vpc_zone_identifier = [for subnet in aws_subnet.private : subnet.id]

  launch_template {
    id      = aws_launch_template.web_server.id
    version = "$Latest"
  }

  health_check_type     = "ELB"
  health_check_grace_period = 300
  target_group_arns     = [aws_lb_target_group.main.id]

  tag {
    key   = "Name"
    value = "WebServer-Instance"
    propagate_at_launch = true
  }
}
```



The Engine: Compute & Auto Scaling

- The core workload is handled by EC2 instances defined in a Launch Template. This template specifies the use of the latest Amazon Linux 2 AMI and a t2.micro instance type to stay within the AWS Free Tier. It also includes a `user_data` script that runs on first boot to automatically install and start the web server.
- An Auto Scaling Group (ASG) manages the lifecycle of these EC2 instances. It is configured to maintain a desired capacity of two instances running across multiple Availability Zones, ensuring the application is both resilient and can automatically scale to handle changes in user traffic.

Load Balancer and Health Checks

The screenshot displays the AWS Management Console interface for an Application Load Balancer (ALB) target group. The top navigation bar shows the AWS logo, search bar, and region (Europe (Frankfurt)). The main content area is titled 'Target groups (1/1)' and includes a 'Create target group' button. Below this, a table lists the target groups, with 'web-app-target-group' selected. The 'Targets' tab is active, showing a list of registered targets. The table has columns for Instance ID, Name, Port, Zone, Health status, Health status details, Admin..., and Overri... The two targets are both marked as 'Healthy'.

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...	Overri...
i-0f8d0b767d31e8e5a	WebServer-Ins...	80	eu-central-1b (...)	Healthy	-	No override	No overri...
i-0a50a54876be15aad	WebServer-Ins...	80	eu-central-1a (...)	Healthy	-	No override	No overri...

Application Load Balancer Configuration and Health Monitoring

- The Application Load Balancer (ALB) distributes incoming HTTP/HTTPS traffic across multiple EC2 instances in the Auto Scaling Group, ensuring high availability and fault tolerance.
- Health checks are configured on the ALB to monitor the status of registered targets by periodically sending requests to a specified health endpoint (e.g., /health).
- Targets responding with HTTP 200 OK status are marked as 'healthy' and continue to receive traffic; unhealthy instances are automatically removed from the load balancer rotation.
- Terraform scripts automate the ALB creation, listener setup, and health check configuration, enabling consistent and repeatable deployments.
- The 'healthy' status screenshot from the AWS console demonstrates all targets passing health checks, confirming the load balancer is correctly routing traffic to functioning instances.

Final Working Webpage



CloudFront Webpage Delivery

- The 'Hello World' webpage is successfully deployed and served via AWS CloudFront, ensuring global low-latency content delivery through edge locations.
- CloudFront acts as a Content Delivery Network (CDN), caching static assets closer to end users to reduce load times and improve availability.
- The origin for CloudFront is the Application Load Balancer, which routes requests to healthy EC2 instances in the Auto Scaling Group.
- This setup enhances fault tolerance and scalability, as CloudFront automatically routes traffic to the nearest edge location and handles failover efficiently.
- The screenshot demonstrates a fully functional webpage with minimal latency, confirming the correctness of the CDN configuration and deployment pipeline.

Source Code Repository

GitHub Repository Overview

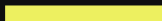
- Repository URL:
<https://github.com/siliconsagenerd/iu-cloud-programming-portfolio>
- Contains Terraform scripts for automated infrastructure provisioning on AWS.
- Includes application code for the 'Hello World' webpage served through CloudFront.
- README file documents project setup instructions and deployment steps.
- Branching strategy follows Git best practices for version control and collaboration.

Repository Structure and Key Files

- /terraform: Contains all Terraform configuration files defining VPC, subnets, EC2, ALB, and other AWS resources.
 - Root Directory (/): Contains all Terraform configuration files (.tf), logically separated by function (e.g., network.tf, security.tf, compute.tf).
 - README.md: Provides detailed instructions on the project's architecture, setup requirements, and deployment steps.
 - .gitignore: Ensures sensitive files, such as local Terraform state (.tfstate) and provider lock files, are not committed to version control.
-

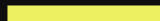
Conclusion and Reflection

Project Outcomes



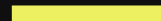
This project successfully implemented a multi-tier AWS architecture featuring a custom VPC, secure subnets, and an EC2 Auto Scaling Group behind an Application Load Balancer. High availability was achieved using Terraform for automated provisioning, and a "Hello World" webpage was deployed globally via CloudFront. The entire project was version-controlled using GitHub.

Challenges Faced



A key technical challenge was diagnosing the initial 502 Bad Gateway error. This was resolved by implementing a NAT Gateway and correctly configuring the private subnet's route table. This fix was crucial, as it provided the necessary internet access for the EC2 instances' startup scripts to complete successfully.

Key Learnings



This project provided significant proficiency in Infrastructure as Code (IaC) principles using Terraform for repeatable deployments. It also improved my understanding of AWS networking, including the critical roles of VPCs, subnets, and gateways, and gave me hands-on experience with compute scaling and health checks.